
"Walk me through your design process."

Started with the equations of motion, reduced to three states, linearised at 10 m/s, checked controllability and observability, designed LQR with integral action on the augmented system, then designed the Luenberger observer with faster poles. Combined them as an output-feedback controller and validated through simulation.

"Why did you choose LQR over pole placement?"

Pole placement requires you to directly specify where the closed-loop poles go, which isn't intuitive. LQR lets you specify performance objectives — how much you care about state deviation versus control effort — through the Q and R weights, and it finds the optimal poles automatically. More systematic and easier to tune.

"How did you compute the controller gain?"

MATLAB's `dlqr` solves the discrete algebraic Riccati equation on the augmented system and returns the gain matrix directly. The augmented system includes the integrator state so the gain comes out as $[K_x \mid K_i]$ in one step.

"Why do you need the integrator?"

LQR alone drives the state deviation to zero but can't reject constant disturbances or track a non-zero reference with zero error. The integrator accumulates the velocity error over time and keeps building the control signal until the error is gone.

"What is anti-windup and why did you include it?"

During the ramp from rest the integrator accumulates a large value because the error is nonzero for a long time. When the reference flattens the integrator unwinds, causing overshoot. Clamping it at ± 200 limits how much it can accumulate without affecting steady-state performance since the clamp is inactive at cruise.

"How did you design the observer?"

Pole placement using MATLAB's `place` function applied to the transposed system — that's the

duality trick. Chose three poles at 0.45, 0.55 and 0.65 in the discrete domain, faster than the controller poles so the estimates converge before the feedback acts on them.

"Why does the observer need to be faster than the controller?"

The separation principle assumes the observer error has settled by the time the controller is acting on the state estimates. If they're the same speed the estimates are still inaccurate when the controller uses them, which degrades performance and can cause instability.

"Can the observer reconstruct the coupler displacement from just velocity

measurements?" Yes — observability being full rank guarantees it. The coupler displacement drives the coupling force which affects the velocity derivatives, so sustained observation of the velocities carries enough information to infer r uniquely. The CA and CA^2 rows of the observability matrix bring those r -dependent terms into the observable subspace.

"Why numerical linearisation instead of analytical?"

The dynamics are personalised through a protected black box so the exact parameters aren't accessible. We derived the analytical Jacobian from the nominal equations to understand the structure, but used finite differences on the black box directly to get the actual matrices for the design.

"What causes the steady-state oscillations?"

The student-ID parameterisation introduces a sinusoidal disturbance. Integral action rejects constant disturbances but not persistent sinusoids — for that you'd need an internal model tuned to the disturbance frequency, like a notch filter.

"Does the controller work far from the operating point?"

Reasonably well — the studies at 8 and 15 m/s both achieved zero steady-state error. But the linear model is only strictly valid near 10 m/s. The further you operate from trim the more the nonlinear terms corrupt the model, and eventually the controller would fail. Gain scheduling would fix that properly.

"What would happen if you used the same poles for the observer and controller?"

The separation principle still holds mathematically but in practice the observer error and control action compete on the same timescale. The state estimates haven't converged when the controller starts acting on them, which degrades transient performance and risks instability.

"What if you increased R significantly?"

Larger R penalises control effort more heavily, so the LQR produces smaller gains. The system responds more slowly, tracking lag increases, and the ramp following gets worse. The steady-state error stays zero because the integrator still eliminates it, just more slowly.

"How is the gain of the controller computed in the script?"

`dlqr(Aaug, Baug, Q, R)` on the augmented discrete system. It solves the discrete algebraic Riccati equation and returns the combined gain `[Kx | Ki]` in one call. `Kx` feeds back the state estimate, `Ki` feeds back the integrated error.

"How is the gain of the observer computed in the script?"

`place(Ad', Cd', obsPoles)'` — pole placement using the duality trick. Transposing the system turns the observer design into an equivalent state-feedback problem, `place` finds the gain, then you transpose back to get `Ld`.

"How do you set the initial conditions?"

In `System_Simulator`, `x0 = [0; 0; 0]` for the main simulation — starting from rest. For Study A, `x0A = xbar + [0; -0.5; 0.05]` — a small perturbation from trim. The observer initial state `xhat0` is set to zeros for the main simulation, and for Study A it's set to the known perturbation `dev = x0A - xbar` so the observer doesn't start blind.

"Where do you define and use the model parameters?"

The physical parameters live inside `Train_Dynamics.p` — we can't access them directly. In our code, the operating speed `vDesign = 10`, sample time `Ts = 0.02`, and student ID are defined at the top of the master script and passed through to every function. The Q and R matrices are defined inside `designTrain3StateController`.

"If LQR was picked, how do you change the closed-loop dynamics?"

Adjust the Q and R matrices. Increasing Q relative to R makes the controller more aggressive — faster response, more control effort, poles move further inside the unit circle. In the code the `QScale` parameter multiplies the entire Q matrix, so increasing it speeds up the response across all states simultaneously. You can also selectively increase individual diagonal entries of Q to prioritise specific states.

"How do you change the observer eigenvalues?"

Change the `obsPoles` vector in `designTrain3StateController`. Currently `linspace(0.45, 0.65, 3)` gives three poles at 0.45, 0.55, 0.65. Moving them closer to zero makes the observer faster but more sensitive to noise. Moving them closer to the unit circle makes it slower but smoother.

"Does it work for all initial conditions?"

No. The controller is designed around a linear model valid near 10 m/s. For small deviations from trim it works well — Study A shows recovery from a perturbation in under a second. Starting from rest is a large deviation from the operating point, and the nonlinear model differs significantly from the linear approximation at low speeds. It still works in practice because the feedforward provides the bulk of the required thrust and the integrator handles the rest, but there's no formal stability guarantee far from trim. A gain-scheduled controller would be needed to guarantee performance across the full speed range.

"Does it work for all sampling rates?"

No. The fastest open-loop pole is at -21.54 rad/s in continuous time, requiring $T_s < 0.046$ s by Nyquist. At $T_s = 0.02$ s we're sampling at 50 Hz, comfortably above this. If T_s were increased toward 0.046 s the discrete controller would struggle to track the fast coupler dynamics and performance would degrade. Beyond that it would become unstable. On the other side, very fast sampling introduces no real benefit and eventually hits numerical precision issues in the ZOH discretisation, though this is less of a practical concern.

"What if you used the wrong order of L or K elements?"

The gains would be applied to the wrong states. For example if $K_x = [-5374, 13121, 14812]$ but the elements were swapped to $[13121, -5374, 14812]$, the large positive gain would multiply the coupler displacement instead of the velocity, likely destabilising the system. For L, wrong

ordering means the observer corrects the wrong state estimates based on the measurement innovation, so the estimates diverge rather than converge.

"What if you used K instead of $-K$?"

Positive feedback instead of negative. The controller would amplify deviations rather than suppress them — the system would be driven away from the setpoint and become unstable immediately.

"What are your performance specifications and did you achieve them?"

The primary specification was zero steady-state velocity error — achieved, 0.0046 m/s which is effectively zero given the persistent sinusoidal disturbance. Secondary specifications were stable tracking across the ramp and reasonable control effort. The ramp is tracked with small lag and modest overshoot at the transition. Control effort peaks at ~19000 N during acceleration which is physically large but within the unconstrained model. All three simulation studies maintain stability and zero steady-state error, confirming the design criteria are met across a range of operating conditions.